STILL WRITING SQL LIKE THIS?

- ✗ No version control
- ✗ No tests
- ✗ No documentation

dbt FIXES ALL THREE.

- Version Controlled
- Tested (Data Quality)
- Auto Documentation
- Lineage Tracking

SQL, engineered.

What is dbt?

dbt = SQL + software engineering practices.

SQL transformations as version-controlled code

Tests on data quality

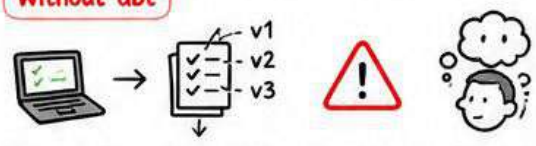
Auto-generated documentation

Lineage tracking across pipeline

Compiles to native warehouse SQL (Snowflake / BigQuery / Databricks)

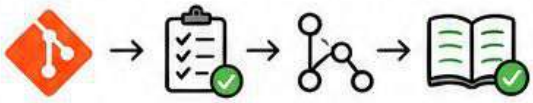
1 The problem it solves

Without dbt



SQL in random notebooks | No idea which version is "right" | Issues found in dashboards (not pipeline) | Docs in someone's head

With dbt



SQL in Git, versioned | Tests catch issues early (CI/CD) | Lineage shows what depends on what | Docs always generated & up to date

2 Core concepts

- **Models** — SQL queries as tables/views
- **Sources** — Raw data your models build on
- **Tests** — Assertions on data (uniqueness, freshness, etc.)
- **Macros** — Reusable SQL snippets
- **Snapshots** — SCD Type 2 done elegantly
- **Seeds** — Small static reference tables

3 Why it's winning in 2026

dbt shows up in **7 of 10** Data Engineer job postings.

Senior DEs use it daily.

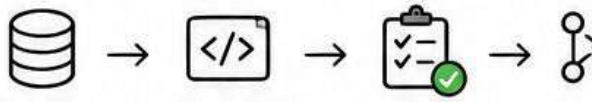
Companies are migrating from custom Spark transforms to dbt.

"I know dbt" in 2026 is what "I know SQL" used to mean.

4 How to get started

1. Install dbt-core (it's free)
2. Connect to your warehouse
3. Convert ONE existing SQL query into a dbt model
4. Add a uniqueness test on the key
5. Run dbt build, see it pass

That's it. You're a dbt user.



dbt Better SQL. Better Data.